



Linux for Beginners

A Practical Guide

Chapter 3 of 12

Files & Directories: Creating, Moving, and Deleting

Contents

3.1	The Linux Filesystem Revisited
3.2	Creating Files — touch and Redirection
3.3	Creating Directories — mkdir
3.4	Viewing File Contents — cat, less, head, tail
3.5	Copying Files and Directories — cp
3.6	Moving and Renaming — mv
3.7	Deleting Files and Directories — rm and rmdir
3.8	Wildcards: Working with Multiple Files at Once
3.9	Finding Files — find and locate
3.10	Putting It All Together: A Practical Workflow
3.11	Essential Commands: Quick Reference
3.12	Chapter Summary

Chapter 3

Files & Directories: Creating, Moving, and Deleting

3.1 The Linux Filesystem Revisited

In Chapter 2 you learned to navigate the filesystem — to move around and see what's there. This chapter is about *changing* it: creating new files and directories, copying and moving things around, and deleting what you no longer need.

Everything in Linux is a file. Directories are files that contain other files. Even devices like your hard drive and keyboard are represented as files. This consistency means the same small set of commands — `touch`, `mkdir`, `cp`, `mv`, `rm` — handles almost everything you'll ever need to do.

Before we start, let's set up a safe sandbox directory to practice in:

```
$ mkdir ~/linux-practice
$ cd ~/linux-practice
$ pwd
/home/ashik/linux-practice
```

bash

All examples in this chapter assume you're working inside `~/linux-practice`. Nothing you do here will affect the rest of your system.

3.2 Creating Files — touch and Redirection

There are several ways to create a file in Linux. The simplest is `touch`.

Using touch

`touch` was originally designed to update a file's timestamp (its "last modified" time), but its most common use is creating an empty file:

```
$ touch notes.txt
$ touch file1.txt file2.txt file3.txt # create multiple at once
$ ls
file1.txt file2.txt file3.txt notes.txt
```

bash

Creating files with content using redirection

The **output redirection operator** `>` sends the output of a command into a file instead of printing it on screen. If the file doesn't exist, it creates it. If it does exist, it **overwrites** it completely.

```
$ echo "Hello, Linux!" > greeting.txt
$ echo "This is my first file." > notes.txt
$ cat greeting.txt
Hello, Linux!
```

bash

The **append operator** `>>` adds to the end of a file without destroying existing content:

```
$ echo "Line one" > myfile.txt
$ echo "Line two" >> myfile.txt
$ echo "Line three" >> myfile.txt
$ cat myfile.txt
Line one
Line two
Line three
```

bash

Operator	Behaviour	Example
<code>></code>	Redirect output to file (overwrites)	<code>echo 'hi' > file.txt</code>
<code>>></code>	Append output to file (preserves)	<code>echo 'hi' >> file.txt</code>
<code><</code>	Read input from a file	<code>sort < names.txt</code>

TIP

The difference between `>` and `>>` is critical. Using `>` on an existing file silently destroys all its contents. When in doubt, use `>>` to be safe.

3.3 Creating Directories — `mkdir`

`mkdir` (**make directory**) creates one or more new directories.

```
$ mkdir projects
$ mkdir docs reports archives      # create three at once
$ ls
archives  docs  projects  reports
```

bash

Creating nested directories with `-p`

By default, `mkdir` will fail if any part of the path doesn't exist yet. The `-p` flag (**p**arents) creates all intermediate directories automatically:

```
bash
# Without -p this would fail if 'projects' doesn't exist:
$ mkdir projects/webserver/config
mkdir: cannot create directory 'projects/webserver/config': No such file or directory

# With -p it creates the full path:
$ mkdir -p projects/webserver/config
$ mkdir -p projects/webapp/src projects/webapp/tests
$ find projects/ -type d
projects/
projects/webapp/
projects/webapp/src/
projects/webapp/tests/
projects/webserver/
projects/webserver/config/
```

TIP

Always use `mkdir -p` when creating multi-level paths. It's safe to run even if the directories already exist — it won't overwrite anything.

3.4 Viewing File Contents — `cat`, `less`, `head`, `tail`

Before editing or moving a file, you often need to read it. Linux gives you several tools depending on how much of the file you want to see.

cat — dump the whole file

`cat` (**c**on**c**atenate) prints the entire contents of a file to the screen at once. It's fast and simple, but impractical for large files:

```
bash
$ cat notes.txt
This is my first file.

$ cat -n notes.txt      # -n adds line numbers
 1 This is my first file.
```

less — read large files comfortably

`less` opens a file in a scrollable viewer — ideal for log files, man pages, or any file longer than your screen. It never modifies the file.

```
$ less /var/log/syslog
```

bash

```
# Navigation inside less:
# Arrow keys / j / k      scroll line by line
# Space / b              scroll page down / up
# G                      jump to end of file
# g                      jump to beginning
# /searchterm            search forward
# n / N                  next / previous match
# q                      quit
```

head and tail — peek at the start or end

`head` shows the first 10 lines of a file (or `N` lines with `-n N`). `tail` shows the last 10. Both are invaluable for inspecting log files:

```
$ head notes.txt           # first 10 lines
$ head -n 5 notes.txt      # first 5 lines
$ tail notes.txt           # last 10 lines
$ tail -n 20 /var/log/syslog # last 20 lines of a log

# Live log monitoring — updates in real time:
$ tail -f /var/log/syslog
```

bash

Command	Best For	Key Option
cat	Small files, quick checks, combining files	-n (add line numbers)
less	Large files, logs, reading comfortably	-N (line numbers in viewer)
head	Checking the start / header of a file	-n N (show N lines)
tail	Checking the end / latest log entries	-f (follow, live updates)

3.5 Copying Files and Directories — cp

`cp` copies files or directories from one location to another. The original is left untouched.

Copying files

```
# cp source destination
$ cp notes.txt notes_backup.txt           # copy in same directory
$ cp notes.txt docs/                      # copy into docs folder
$ cp notes.txt docs/notes_copy.txt        # copy and rename
$ cp file1.txt file2.txt docs/            # copy multiple files to a dir
```

Copying directories

By default, `cp` refuses to copy directories. You need the `-r` flag (recursive) to copy a directory and all its contents:

```
$ cp -r projects/ projects_backup/        # copy entire directory tree
$ cp -r docs/ /tmp/docs_backup/           # copy to a different location
```

Useful `cp` options

Option	Meaning
<code>-r</code> or <code>-R</code>	Recursive — required for copying directories
<code>-i</code>	Interactive — ask before overwriting an existing file
<code>-u</code>	Update — only copy if source is newer than destination
<code>-v</code>	Verbose — print each file as it's copied
<code>-p</code>	Preserve — keep original timestamps and permissions
<code>--backup</code>	Make a backup of the destination file before overwriting

TIP

Use `cp -i` when you're not sure whether a destination file already exists. It will prompt 'overwrite file.txt?' instead of silently replacing it.

3.6 Moving and Renaming — `mv`

`mv` does two things depending on how you use it: it **moves** a file to a different directory, or it **renames** a file within the same directory. Unlike `cp`, `mv` does not leave a copy behind — the original is gone from its source location.

Renaming a file

```
# If source and destination are in the same directory, mv renames:
$ mv notes.txt journal.txt          # notes.txt is now journal.txt
$ mv old_name.txt new_name.txt
```

bash

Moving a file to a different directory

```
$ mv journal.txt docs/              # move into docs/
$ mv docs/journal.txt .            # move back to current dir
$ mv file1.txt file2.txt reports/  # move multiple files at once
```

bash

Moving and renaming at the same time

```
# Move AND rename in one command:
$ mv notes.txt docs/journal_2024.txt
```

bash

Moving directories

Unlike `cp`, `mv` does not need `-r` to move directories. It moves the entire directory tree in one atomic operation:

```
$ mv projects/ old_projects/      # rename directory
$ mv old_projects/ archives/      # move into archives/
```

bash

TIP

`mv -i` will prompt before overwriting an existing file at the destination — good practice when moving important files.

3.7 Deleting Files and Directories — `rm` and `rmdir`

This is the most dangerous section in the chapter. Linux does not have a Recycle Bin or Trash folder from the command line. When you delete a file with `rm`, it is **gone immediately and permanently**. There is no undo.

Deleting files with `rm`

```
$ rm file1.txt                    # delete a single file
$ rm file1.txt file2.txt file3.txt # delete multiple files
$ rm -i important.txt             # -i asks for confirmation first
```

bash

Deleting empty directories with `rmdir`

`rmdir` removes a directory, but only if it is completely empty:

```
$ rmdir empty_folder          # works only if folder is empty
$ rmdir docs/                 # fails if docs/ has any files
```

bash

Deleting directories and their contents with `rm -r`

To delete a directory along with everything inside it, use `rm -r` (recursive):

```
$ rm -r old_projects/         # delete directory and all contents
$ rm -ri reports/             # ask for confirmation on each file
```

bash

WARNING

WARNING: `rm -rf` is the most dangerous command a beginner will encounter. The `-f` flag (force) suppresses all warnings and confirmation prompts. Running `rm -rf /` or `rm -rf ~` would destroy your entire system or home directory with no warning and no recovery. Never copy-paste `rm -rf` commands without reading them character by character.

Safe deletion habits

- Always use `rm -i` when you're unsure — it asks before each deletion
- Run `ls` on the target first to see exactly what will be deleted
- Use `echo rm -r folder/` to preview the command without running it
- Consider moving files to a temporary folder before permanently deleting them

3.8 Wildcards: Working with Multiple Files at Once

Wildcards (also called **glob patterns**) let you match multiple filenames with a single expression. They work with almost every command — `ls`, `cp`, `mv`, `rm`, and more.

Wildcard	Matches	Example
<code>*</code>	Any number of any characters (including none)	<code>*.txt</code> — all <code>.txt</code> files
<code>?</code>	Exactly one character	<code>file?.txt</code> — <code>file1.txt</code> , <code>fileA.txt</code>
<code>[abc]</code>	Exactly one character from the set	<code>[aeiou]*</code> — files starting with a vowel
<code>[a-z]</code>	Exactly one character in the range	<code>[0-9]*</code> — files starting with a digit
<code>[!abc]</code>	Any character NOT in the set	<code>[!0-9]*</code> — files not starting with digit

Wildcards in practice

```
# List all .txt files:
$ ls *.txt

# Copy all .log files to archives/:
$ cp *.log archives/

# Delete all temporary files:
$ rm *.tmp

# Move files matching a pattern:
$ mv report_202?.txt reports/

# List files starting with 'file' followed by a single digit:
$ ls file[0-9].txt
```

bash

TIP

Before using wildcards with `rm`, test them with `ls` first. `ls *.log` shows you exactly which files match before `rm *.log` deletes them.

3.9 Finding Files — `find` and `locate`

As your filesystem grows, you'll need to search for files. Linux provides two main tools for this, each with different strengths.

`find` — search in real time

`find` searches the filesystem live, looking at actual files on disk. It's flexible and powerful but can be slow on large directory trees.

```
# Basic syntax: find <where> <what>

# Find all .txt files in current directory and below:
$ find . -name '*.txt'

# Find files with exact name:
$ find /home -name 'notes.txt'

# Find only directories named 'config':
$ find / -type d -name 'config'

# Find files modified in the last 7 days:
$ find . -mtime -7

# Find files larger than 100MB:
$ find / -size +100M

# Find and delete (use with extreme care):
$ find . -name '*.tmp' -delete
```

bash

locate — search from a database

Locate searches a pre-built index of the filesystem — much faster than `find`, but the index is only updated once a day (or when you run `sudo updatedb`):

```
$ locate notes.txt          # find any file named notes.txt
$ locate '*.pdf'            # find all PDF files
$ sudo updatedb             # manually refresh the database
```

bash

Tool	Speed	Up to date?	Best For
find	Slower (live scan)	Always current	Precise searches, recent files
locate	Very fast (database)	Only as fresh as index	Quick searches by filename

3.10 Putting It All Together: A Practical Workflow

Let's combine everything from this chapter into a realistic scenario. Suppose you're organising a project directory that's become messy:

```
bash
# 1. See what we're working with:
$ ls -lah ~/linux-practice

# 2. Create an organised structure:
$ mkdir -p organised/{docs,scripts,logs,archive}

# 3. Move text files to docs:
$ mv *.txt organised/docs/

# 4. Copy log files to logs (keep originals):
$ cp *.log organised/logs/

# 5. Verify the result:
$ ls -R organised/

# 6. Read the main notes file to confirm it moved correctly:
$ cat organised/docs/notes.txt

# 7. Delete the original log files now that we have copies:
$ rm *.log

# 8. Confirm cleanup:
$ ls
organised/
```

Notice the pattern: **look before you leap**. Run `ls` before and after every significant change. Read files with `cat` or `less` to confirm they're what you expect before deleting originals. This habit prevents almost every accidental data loss.

3.11 Essential Commands: Quick Reference

Command	What It Does	Common Usage
<code>touch</code>	Create an empty file (or update timestamp)	<code>touch notes.txt</code>
<code>echo 'x' > file</code>	Write text to a file (overwrites)	<code>echo 'hi' > file.txt</code>
<code>echo 'x' >> file</code>	Append text to a file	<code>echo 'hi' >> file.txt</code>
<code>mkdir</code>	Create a directory	<code>mkdir projects</code>
<code>mkdir -p</code>	Create nested directories	<code>mkdir -p a/b/c</code>
<code>cat</code>	Print entire file to screen	<code>cat notes.txt</code>
<code>cat -n</code>	Print file with line numbers	<code>cat -n script.sh</code>

Command	What It Does	Common Usage
<code>less</code>	Scroll through a file	<code>less /var/log/syslog</code>
<code>head</code>	Show first 10 lines	<code>head -n 5 notes.txt</code>
<code>tail</code>	Show last 10 lines	<code>tail -f app.log</code>
<code>cp src dst</code>	Copy a file	<code>cp file.txt backup.txt</code>
<code>cp -r src dst</code>	Copy a directory recursively	<code>cp -r docs/ docs_bak/</code>
<code>cp -i src dst</code>	Copy with overwrite confirmation	<code>cp -i data.csv reports/</code>
<code>mv src dst</code>	Move or rename a file/directory	<code>mv old.txt new.txt</code>
<code>rm</code>	Delete a file permanently	<code>rm temp.txt</code>
<code>rm -i</code>	Delete with confirmation prompt	<code>rm -i important.txt</code>
<code>rm -r</code>	Delete directory and all contents	<code>rm -r old_project/</code>
<code>rmdir</code>	Delete an empty directory	<code>rmdir empty_folder/</code>
<code>find . -name</code>	Search for files by name	<code>find . -name '*.txt'</code>
<code>locate</code>	Fast filename search from index	<code>locate notes.txt</code>

3.12 Chapter Summary

- ✓ `touch` creates empty files. The `>` operator creates files with content; `>>` appends without overwriting.
- ✓ `mkdir` creates directories. Use `-p` to create nested paths in one command.
- ✓ `cat` dumps a whole file. `less` lets you scroll. `head` and `tail` show the start and end.
- ✓ `cp` copies files; requires `-r` for directories. The original is preserved.
- ✓ `mv` moves or renames — no `-r` needed for directories. The source is removed.
- ✓ `rm` deletes permanently with no undo. Always use `-i` when unsure. **`rm -rf` is dangerous.**
- ✓ **Wildcards** (`*`, `?`, `[]`) let you act on many files at once. Test with `ls` before using with `rm`.
- ✓ `find` searches live and is precise. `locate` searches a database and is fast.
- ✓ The golden rule: **Is before and after every significant change.** Read files before deleting originals.

What's Coming in Chapter 4

You can now create, read, copy, move, and delete anything on the filesystem. But not all files are equal — Linux has a detailed permission system that controls exactly *who* can do what with each file. Chapter 4 covers:

- Users and groups — how Linux identifies who you are
- The permission model: read, write, execute for owner, group, and others
- Reading the `drwxr-xr-x` permission string
- Changing permissions with `chmod`
- Changing ownership with `chown`
- The root user and `sudo` — when and why to use them

Permissions are the foundation of Linux security. Chapter 4 is where Linux starts to make real sense.

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.
